

Penetration Testing

U2U (DeFi)

Table of Contents

Executive Summary	4
Project Context	4
Penetration Test scope	7
Security Rating	8
Code Quality	9
Penetration Test Resources	9
Dependencies	9
Severity Definitions	10
Penetration Test Findings	11
Centralisation	30
Conclusion	31
Our Methodology	32
Disclaimers	34
About Hashlock	35

CAUTION

THIS DOCUMENT IS A SECURITY PENETRATION TEST REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE THAT COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR THE USE OF THE CLIENT.

Executive Summary

The U2U team partnered with Hashlock to conduct a security Penetration Test of their U2U Super App & Mobile wallet code base. Hashlock manually and proactively reviewed the code to ensure the project's team and community that the deployed contracts were secure.

Project Context

The U2U Network is a decentralized blockchain platform designed to support scalable real-world applications, leveraging a Direct Acyclic Graph (DAG)-based architecture and EVM compatibility for fast, secure, and highly efficient operations. It aims to empower decentralized services, with innovations like DePIN (Decentralized Physical Infrastructure Networks) and modular subnet technology, allowing for high throughput (up to 72,000 TPS) and rapid transaction finality (650ms). U2U focuses on enabling decentralized privacy networks, IoT integration, decentralized storage, and digital identity solutions. This infrastructure aims to drive a decentralized, scalable future for Web3 applications.

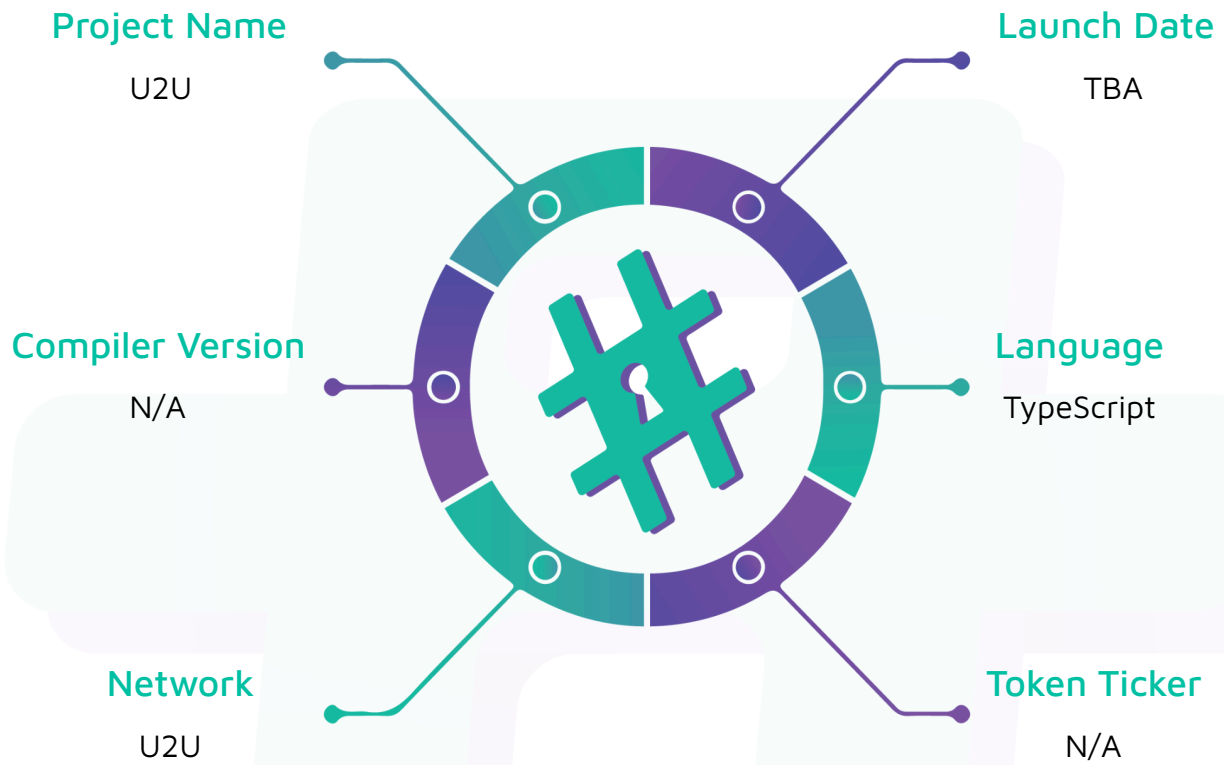
Project Name: U2U

Compiler Version: N/A

Website: <https://u2u.xyz/>

Logo:



Visualised Context:

Project Visuals:



Penetration Test scope

At Hashlock, we executed a comprehensive penetration test on the U2U Super App & Mobile wallet code base. Our scope included a detailed security assessment, where we rigorously examined the code to identify vulnerabilities and assess overall efficiency. The testing process involved a meticulous manual review, supplemented by advanced software-assisted techniques, to ensure thorough analysis and accuracy.

Description	U2U Mobile Wallet code base
Platform	Typescript
Penetration Test Date	September - October, 2024
Repository	https://github.com/unicornultrafoundation/u2u-mobile-wallet
GitHub Commit Hash	5dc294a1abdd7b123286d6d110df4e22ea0b1618

Security Rating

After Hashlock's Penetration Test, we found the code base to be **"Secure"**. The code follows simple logic, with correct and detailed ordering.



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on-chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Penetration Test Findings](#) section.

We initially identified some significant vulnerabilities that have since been addressed.

Hashlock found:

1 High-severity vulnerabilities

5 Medium-severity vulnerabilities

3 Low-severity vulnerabilities

6 QA issues

Caution: *Hashlock's Penetration Tests do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Code Quality

This Penetration Test scope involves the code bases of the U2U Super App & Mobile wallet, as outlined in the Penetration Test Scope section. All contracts, libraries, and interfaces mostly follow standard best practices to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was required.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Penetration Test Resources

We were given the U2U Super App & Mobile wallet code base in the form of GitHub access.

As mentioned above, code parts are well-commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments help understand the overall architecture of the protocol.

Dependencies

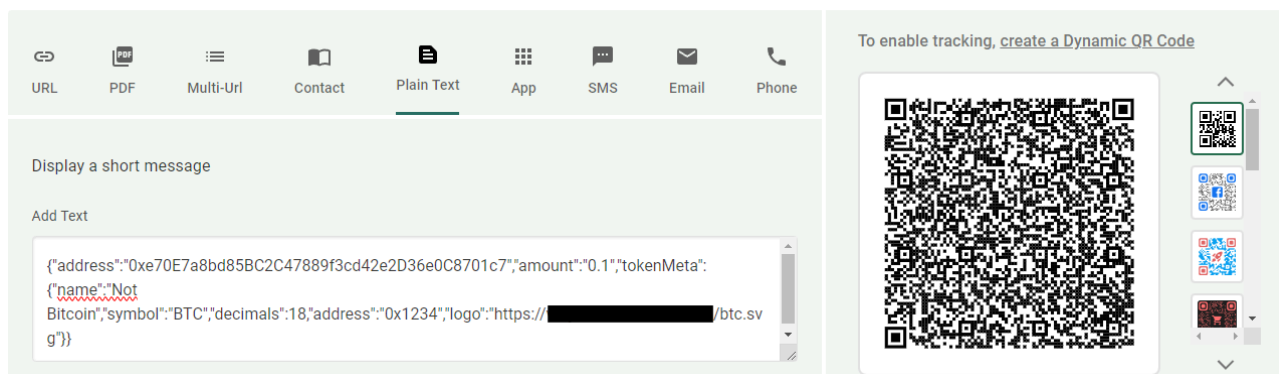
Per our observation, the libraries used in this code base's infrastructure are based on well-known industry-standard open-source projects.

Apart from libraries, its functions are used in external code base calls.

Severity Definitions

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies
QA	Quality Assurance (QA) findings are purely informational and don't impact functionality. These notes help clients improve the clarity, maintainability, or overall structure of the code, ensuring a cleaner and more efficient project. They should be addressed for optimization but are not critical to the system's performance or security.

tokens in the wallet, it was not possible to confirm if the transaction succeeded. Hence, only the UI was manipulated. The successful Proof of Concept would require at least two tokens to be present in the wallet. The following data was put on a QR code:



```
{\"address\": \"0xe70E7a8bd85BC2C47889f3cd42e2D36e0C8701c7\", \"amount\": \"0.1\", \"tokenMeta\": {\"name\": \"Not Bitcoin\", \"symbol\": \"BTC\", \"decimals\": 18, \"address\": \"0x1234\", \"logo\": \"https://[redacted]/btc.svg\"}}}
```

Once scanned, the screen showed a transfer for 0.1 BTC with a BTC symbol loaded from the remote server.

In a real world attack, an attacker would put a cheap token data but address of an expensive one in order to convince a victim to transfer more funds than intended.

Impact

Loss of funds, privacy violation.

Recommendation

Similarly to the DeFi whitelist, a token whitelist should be introduced and loaded by reference only from a U2U-controlled server. Such a list may contain token entries with symbols, logos, decimals, and addresses. A user-controlled data should not be part of the transfer QR code.

Status

Resolved

Medium

[M-01] Backend API - Anyone can see other users blocked contacts

Description

In `src/service/chat.ts` a couple of chat requests are defined, which include blocking/unblocking contacts or viewing conversations. Most of them are secured by signature check but `getBlockedContacts` is not. Therefore, other users can see who blocked who. This is a minor privacy issue, however, since all other endpoints are secured with a signature, this one could also implement such protection, as there is no need for other users to know who blocked who.

Vulnerability Details

The below listing shows a GET request to the wallet backend without a signature requirement

```
GET
/conversations/block?page=1&limit=2&address=0x789d0DBFf754eBCd0e63eC22064384eb160a325B
HTTP/2

Host: sa-backend-dev.u2u.xyz

Cache-Control: max-age=0

Sec-Ch-Ua: "Not;A=Brand";v="24", "Chromium";v="128"

Sec-Ch-Ua-Mobile: ?0

Sec-Ch-Ua-Platform: "Windows"

Accept-Language: en-US,en;q=0.9

Upgrade-Insecure-Requests: 1

User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like
Gecko) Chrome/128.0.6613.120 Safari/537.36

Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng
,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
```

```
Sec-Fetch-Site: none
Sec-Fetch-Mode: navigate
Sec-Fetch-User: ?1
Sec-Fetch-Dest: document
Accept-Encoding: gzip, deflate, br
Priority: u=0, i
```

Response:

```
HTTP/2 200 OK

Date: Sat, 05 Oct 2024 07:08:59 GMT

Content-Type: application/json; charset=utf-8
Content-Length: 21
Access-Control-Allow-Origin: *
Etag: W/"15-OR0s70rrFasfn9uBcIaD2hUmAok"
Cf-Cache-Status: DYNAMIC

Report-To:
{"endpoints":[{"url":"https://a.ne1.cloudflare.com/report/v4?s=6RCsxxg3ou5d0VCkg1co
BnCtVIyL0Fg9%2BvqzwzvtAJIwFhBN9tW8E5Kd0FS1wJm2Xr5I6KYM06B0CQaa4ccbRrJbVUfsxwZ03i7eHkQv
FL2ZYn0jXjaC2cG4J26TV7IVlRJdn9KXqE1XV"}], "group":"cf-ne1", "max_age":604800}

Nel: {"success_fraction":0, "report_to":"cf-ne1", "max_age":604800}

Expect-Ct: max-age=86400, enforce

Referrer-Policy: same-origin

X-Content-Type-Options: nosniff

X-Frame-Options: SAMEORIGIN

X-Xss-Protection: 1; mode=block

Server: cloudflare

Cf-Ray: 8cdb9562ccbcc3e2-WAW
```

```
{"data": [], "count": 0}
```

Impact

A minor privacy issue.

Recommendation

It's recommended that that endpoint be protected with a signature check as well.

Status

Resolved

[M-02] Backend API - Users can register their Firebase notification tokens with arbitrary wallets

Description

The notification service works by matching a wallet address with a Firebase token. However, since there are no signature checks which means that users can register their Firebase tokens with the wallet addresses of other users possibly reading their notifications and incoming messages.

Vulnerability Details

The below listing shows a GET request to the wallet backend without a signature requirement

```
POST /wallets/register-notification-devices HTTP/2
```

```
Host: sa-backend-dev.u2u.xyz
```

```
Content-Type: application/json
```

```
Content-Length: 233
```

```
Accept-Encoding: gzip, deflate, br
```

```
User-Agent: okhttp/4.9.2
```

```
{
  "tokens": [
    "dCxB5YQFRraoNqx-Bwe1Yg:APA91bEG1jq1u0g74ZCb-wqQnH1qPof88XXxyEYUIkJGEpDWRAmcpjsU3cauzK0SpmUaCU70DQ80x9C4zK7H7h9Th1eiXGnrRuCv80d0TQVyzJKinz8Zfo6kW3_7JnhU2ZJ_azGqZi28"
  ],
  "address": "0xe70E7a8bd85BC2C47889f3cd42e2D36e0C8701c7"
}
```

Response:

```
HTTP/2 201 Created

Date: Tue, 08 Oct 2024 13:15:02 GMT

Content-Type: application/json; charset=utf-8

Content-Length: 727

Access-Control-Allow-Origin: *

Etag: W/"2d7-cFTduZVGa1cGI4pZgxv2xtCk1A"

Cf-Cache-Status: DYNAMIC

Report-To:
{"endpoints":[{"url":"https://a.nel.cloudflare.com/report/v4?s=Sb%2BhKdR221b1Uj1XP8ov1RvK%2FS6zmik0x%2Bfmim2ZaLXMAXxQ0RteUEjAJB%2BGiwPixV9sUuKNAu9MXk6YQBg0wGxMz%2BY3kMaBgZLjL9UAJa9uU%2Ftsve5caGjA8bT%2BjZ5k8qP5eFtk5QUx"}],"group":"cf-nel","max_age":604800}]

Nel: {"success_fraction":0,"report_to":"cf-nel","max_age":604800}

Expect-Ct: max-age=86400, enforce

Referrer-Policy: same-origin

X-Content-Type-Options: nosniff

X-Frame-Options: SAMEORIGIN

X-Xss-Protection: 1; mode=block

Server: cloudflare

Cf-Ray: 8cf665b978a0caf9-DUS

{"id":"f283e4a0-e775-41bb-bce0-6d70ce5ea52d","address":"0xe70E7a8bd85BC2C47889f3cd42e2D36e0C8701c7","nickname":"<s>aa</s>","notificationDeviceToken":["dCxB5YQFRraoNqx-Bwe1Yg:APA91bEG1jq1u0g74ZCb-wqQnH1qPof88XXxyEYUIkJGEpDWRAmcpjsU3cauzK0SpmUaCU70DQ80x9C4zK7H7h9Th1eiXGnrRuCv80d0TQVyzJKinz8Zfo6kW3_7JnhU2ZJ_azGqZi28"]}
```



```
g:APA91bEGljqlu0g74ZCb-wqQnHlqPof88XXxyEYUIkJGepDWRAmcpjsU3cauzKOSpmUaCU70DQ80x9C4zK7H
7h9Th1eiXGnrRuCv80d0TQVyzJKinz8Zfo6kW3_7JnhU2ZJ_azGqZi28", "dP2L8ENGRMa1ttR36T08tG:APA9
1bHJLV7zAtiRu-f3MT-cMB7GzYACZyRp0jixtvpvBw1xw_kucJLM8NNA1a57vmg_21DXkuBzk1RhDXvnUQD4MM
EZaaC23Z7Vudd64XuG1Q-pqvRoAeV-E9IUzHfhHPBiylIoLSam"], "createdAt": "2024-09-25T15:18:02.
302Z", "updatedAt": "2024-10-08T13:15:02.437Z", "devices": [{"id": "49358c67-a098-4758-950f
-732333cd0b3e", "deviceId": "dfc20e1bef3b507d", "createdAt": "2024-09-25T15:17:02.970Z", "u
pdatedAt": "2024-09-25T15:17:02.970Z"}]}
```

Impact

Possibly a major privacy issue, however, since notification/messaging systems were not ready at the time it could not be confirmed.

Recommendation

It's recommended that that endpoint be protected with a signature check.

Status

Resolved

[M-03] Backend API - App Flyers header is not validated when registering a new device

Description

A new device can be registered without any authentication or control. It is expected that the X-App-Flyer-Uid header would at least ensure tracking of the device, however, it seems that it is not validated. This could lead to DoS on the database or Spoofing issues in the future if any feature relies on the value of the aforementioned header.

Vulnerability Details

The below listing shows a GET request to the wallet backend with a placeholder in the app flyer header:

```
POST /devices HTTP/2

Host: sa-backend-dev.u2u.xyz

Content-Type: application/json
```

```
X-App-Flyer-Uid: your_appsflyer_uid_here
```

```
Content-Length: 35
```

```
{  
  "deviceId": "aaaaaaaaaaaa"  
}
```

Response:

```
HTTP/2 201 Created
```

```
Date: Sat, 05 Oct 2024 07:08:59 GMT
```

```
Content-Type: application/json; charset=utf-8
```

```
[...]
```

Impact

Unauthenticated users may try to register fictional devices, leading to overloading the database

Recommendation

It's recommended that that endpoint be protected with a signature check as well.

Status

Resolved

[M-04] Backend API - "Devices" endpoint can be spammed

Description

The `SUBMIT_DEVICE_ID_ENDPOINT` which translates to `/devices`, allows anyone to register a device. There is no restriction on it, which may lead to possible spam attempts.

```
POST /devices HTTP/2
```

```
Host: sa-backend-dev.u2u.xyz
```

```
Content-Type: application/json
```

```
X-App-Flyer-Uid: your_appsflyer_uid_here
```

```
Content-Length: 35
```

```
{
  "deviceID": "aaaaaaaaaaaaa"
}
```

Response:

```
HTTP/2 201 Created
```

```
Date: Sat, 05 Oct 2024 07:39:42 GMT
```

```
Content-Type: application/json; charset=utf-8
```

```
Content-Length: 163
```

```
Access-Control-Allow-Origin: *
```

```
Etag: W/"a3-Mwlge8lykTsj33p8MVP0P2isnKs"
```

```
Cf-Cache-Status: DYNAMIC
```

```
Report-To:
```

```
{"endpoints":[{"url":"https://a.nel.cloudflare.com/report/v4?s=IIhrS2t2Ygqb2
PUmwCyLIoQYgbjtNaGtYY3qDEGFkhUqXGEwIljN7ffSf%2FbDb9V0kIjDFZPzTy7hR5olheaUi3hwARN
yEfSoAxo5Z9z0KEMo%2BdSupzwHkaBgKcfzfZojtoLiUx1Ag%2BTR"}],"group":"cf-nel","max_a
ge":604800}
```

```
Nel: {"success_fraction":0,"report_to":"cf-nel","max_age":604800}
```



Expect-Ct: max-age=86400, enforce

Referrer-Policy: same-origin

X-Content-Type-Options: nosniff

X-Frame-Options: SAMEORIGIN

X-Xss-Protection: 1; mode=block

Server: cloudflare

Cf-Ray: 8cd8c2628f87bf49-WAW

```
{"deviceID":"aaaaaaaaaaaaa","wallets":[],"createdAt":"2024-10-05T07:39:42.021Z",  
"updatedAt":"2024-10-05T07:39:42.021Z","id":"cf3d303c-2d8c-451f-9411-08b6c53c5607"}
```

Impact

A lack of limitations allowed automated requests to register unlimited devices, potentially overwhelming the backend. This could lead to performance degradation or even a denial of service (DoS)

Recommendation

Consider implementing a limit per IP, device, or require authorization on this endpoint.

Status

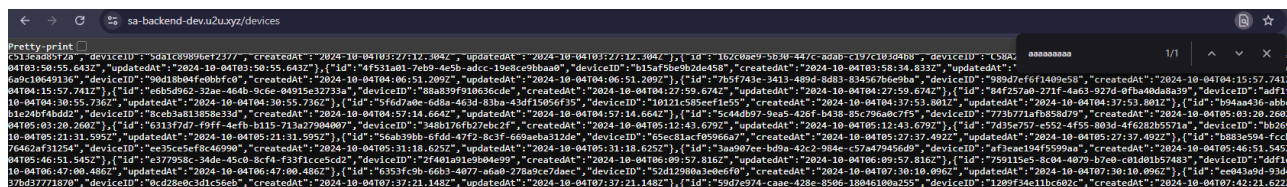
Resolved.

[M-05] Backend API - Undocumented "devices" GET endpoint leaks all devices ever existing

Description

The `SUBMIT_DEVICE_ID_ENDPOINT` which translates to `/devices`, allows anyone to register a device. It was discovered that it also accepts `GET` requests, which were not used in the app code, and it allows for listing all devices ever registered. The latest

created device during the test ("aaaaaaaaaaaa...") was listed at the end, which confirms this is a real-time list. A sample view of the page is presented below, it contains a very large amount of data.



Impact

This can create a privacy risk, as unauthorized parties can access device identifiers and other potentially sensitive metadata.

Recommendation

Consider protecting the above endpoint.

Status

Resolved.

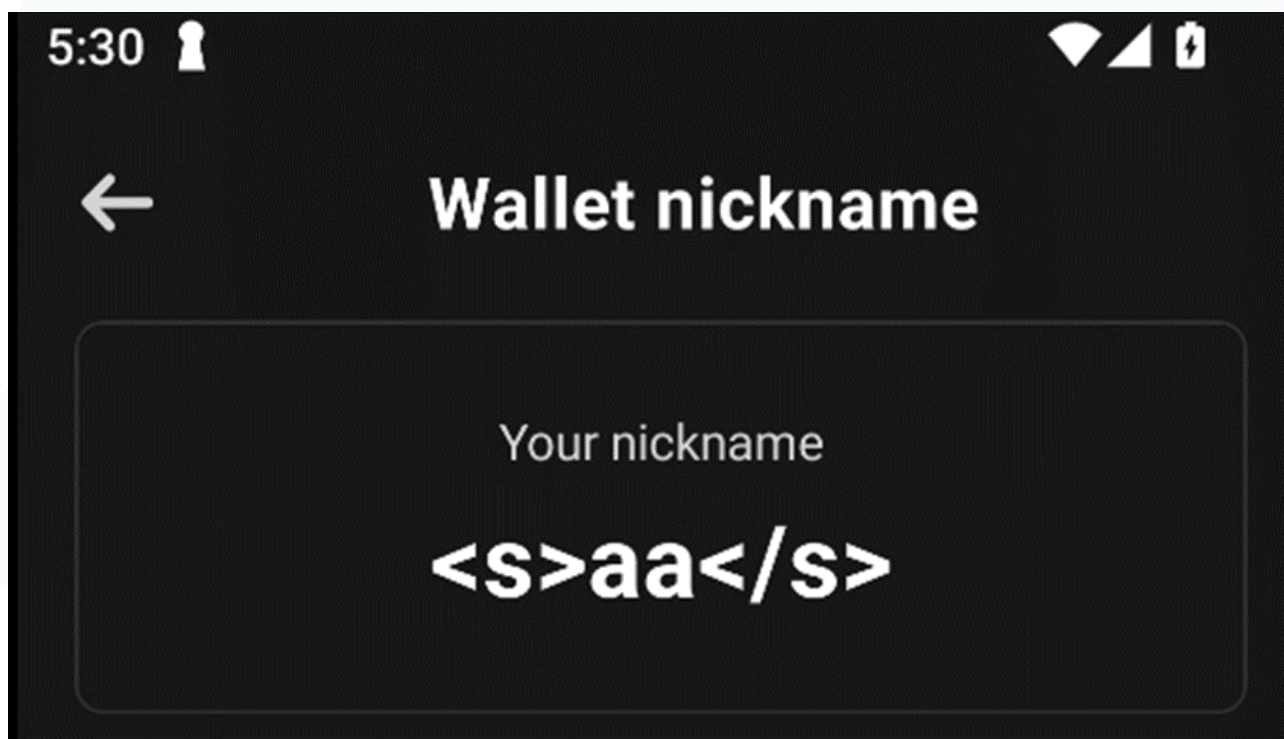
Low

[L-01] U2U Super App - Prohibited characters can still be used in the wallet nickname

Description

The Wallet Nickname screen displays information that special characters are forbidden. This may be because the nickname could be used further on chat or similar features, where this could increase risk of Cross-Site Scripting or HTML injection attacks, potentially threatening user funds.

However, the requirement is never checked and users can create names such as those provided in the screenshot below:



It was discovered also that such a nickname with special characters breaks the transfer functionality, and while it is possible to find a recipient by the nickname, if special characters are present, the transfer fails due to the recipient being empty as per the below screenshot:

←

Send

Send



0.1 U2U

To



EST fee

0 U2U

Max fee

0.00053 U2U >

Wallet

Address 0
0x789d0D...160a325B

Network

 Testnet

Recommendation

Disallow the creation of nicknames with special characters in them. Also, consider checking for duplicate nicknames to avoid confusion/issues in the future if the nicknames are to be used for user-to-user communication.

Status

Resolved

[L-02] U2U Super App - Untrusted websites are loaded before a warning is confirmed

Description

The application explore view allows for browsing any website as long as the user confirms the warning window. The issue is that the underlying website is loaded anyway, and the warning is displayed at the same time.

If a potential exploit or a simple IP-logging website is sent to a user, he should be made aware of the risk before the site is requested and loaded by the app.

Recommendation

Proceed to the site loading routine only after the user confirms the warning. Otherwise do not send any request to an untrusted address.

Status

Resolved

[L-03] U2U Super App - Dependencies should be pinned to exact versions

Description

The application uses external dependencies, some of which are not pinned to an exact version but set to a compatible version (^x.x.x). This can potentially allow for dependency attacks. Similar attacks were seen in the past, for example, Copay Bitcoin Wallet was attacked in such a way. The below listing shows part of the package.json file:

[...]

```
"dependencies": {
```

```
  "@ethersproject/shims": "^5.7.0",
```

```
  "@georstat/react-native-image-cache": "^3.1.0",
```

```
  "@gorhom/bottom-sheet": "^4.5.1",
```

```
"@notifee/react-native": "^7.8.2",  
"@react-native-async-storage/async-storage": "^1.21.0",  
"@react-native-clipboard/clipboard": "^1.11.2",  
"@react-native-community/blur": "^4.4.0",  
"@react-native-community/masked-view": "^0.1.11",  
"@react-native-community/netinfo": "^11.2.1",  
"@react-native-firebase/analytics": "^20.1.0",  
"@react-native-firebase/app": "^20.1.0",  
"@react-native-firebase/app-check": "^20.1.0",  
"@react-native-firebase/crashlytics": "^20.1.0",  
"@react-native-firebase/messaging": "^20.1.0",  
"@react-navigation/bottom-tabs": "^6.5.8",  
"@react-navigation/native": "^6.1.7",  
"@react-navigation/native-stack": "^6.9.13",  
[...]
```

Recommendation

Pin all external dependencies to exact versions (x.x.x) instead of using compatible version ranges (^x.x.x) to mitigate the risk of dependency-based attacks.

Status

Resolved.

QA

[Q-01] U2U Super App - FlyersApp API key leak

Description

It was noticed that the application contains a hardcoded FlyersApp API Key. However, it was not possible to demonstrate any significant impact related to this. Due to that, this is left as a QA issue only. Hardcoding keys or secrets are generally considered insecure, however, some API keys are meant to be public. Due to limited documentation of FlyersApp, it was not clear whether this key should be treated as public.

Recommendation

Verify if the key should be visible to anyone downloading the app.

Status

Resolved

[Q-02] U2U Super App - The app can be installed on a vulnerable unpatched Android version

Description

The `minSdkVersion` allows for the installation of very old versions of Android, which may be insecure and unsupported. Below is the listing from `android/build.gradle` file:

```
buildscript {  
    ext {  
        buildToolsVersion = "33.0.0"  
        minSdkVersion = 21  
        compileSdkVersion = 33  
        targetSdkVersion = 33
```

Recommendation

Consider increasing `minSdkVersion` to at least 23 (Android 6.0) to exclude known vulnerable versions.

Status

Resolved

[Q-03] U2U Super App - Application content is not hidden during split screen**Description**

When entering a split screen mode (switching between applications), the U2U Super App is still visible on its last-used screen. This may lead to a minor privacy violation, for example, a user of the application might not wish to reveal their balances while using a smartphone in a public space.

It is common for many financial/banking applications to implement a protection, so during split screen view the application screen is not visible, and a blank/generic screen is shown instead.

Recommendation

Develop a secure overlay screen that activates when the app loses focus. This screen should not contain any sensitive information and should be displayed immediately when the app is not active. Moreover, consider requiring user re-authentication (e.g., PIN, biometrics) when returning to the app after it has been in the background.

Status

Resolved

[Q-04] U2U Super App - Wallet nickname cannot be changed, but users are not informed about it

Description

The Wallet nickname cannot be changed after setting it initially. However, users are never informed that their initial nickname should be a well-thought decision since they are not able to change it anymore. It is also not possible to change the nickname via backend API, since it displays an error that a wallet already has a nickname, which indicates that this might be the intended design.

This is not a security risk but rather leads to bad user experience, thus should be avoided.

Recommendation

Display a warning to users that the nickname set is a one-time operation or allow for changing it (might be time limited e.g. once per month to avoid potential spoofing issues).

Status

Resolved

[Q-05] U2U Super App - Chat experimental functionality crashes

Description

The u2u super app was tested also on a physical device, a Samsung A10. It was noticed that on attempting to write a message on the chat, the application crashes. On the other hand, the feature was not enabled when running the app inside an emulator. Since this is an experimental feature, it is fully understandable that it may behave in an unstable manner.

Recommendation

Investigate the chat feature.

Status

Resolved

[Q-06] Backend API - Uncertainty about access control check**Description**

The chat endpoints were tested for access control checks in src/service/chat.ts

It was noticed that some getConversationByParticipants and getConversationMessages use

```
myHeaders.append("signature", dto.signature);
```

To add a signature to request headers. However the request to

<https://sa-backend-dev.u2u.xyz/conversations/by-users/0x789d0DBFf754eBCd0e63eC22064384eb160a325B-0x789d0DBFf754eBCd0e63eC22064384eb160a325B>

This leads to responses such as:

```
{ "message": "Cannot GET /conversations/by-users/0x789d0DBFf754eBCd0e63eC22064384eb160a325B-0x789d0DBFf754eBCd0e63eC22064384eb160a325B", "error": "Not Found", "statusCode": 404 }
```

This may mean that either the authentication is not implemented (should be 401) or the endpoint is yet to be implemented.

Recommendation

Investigate the above feature.

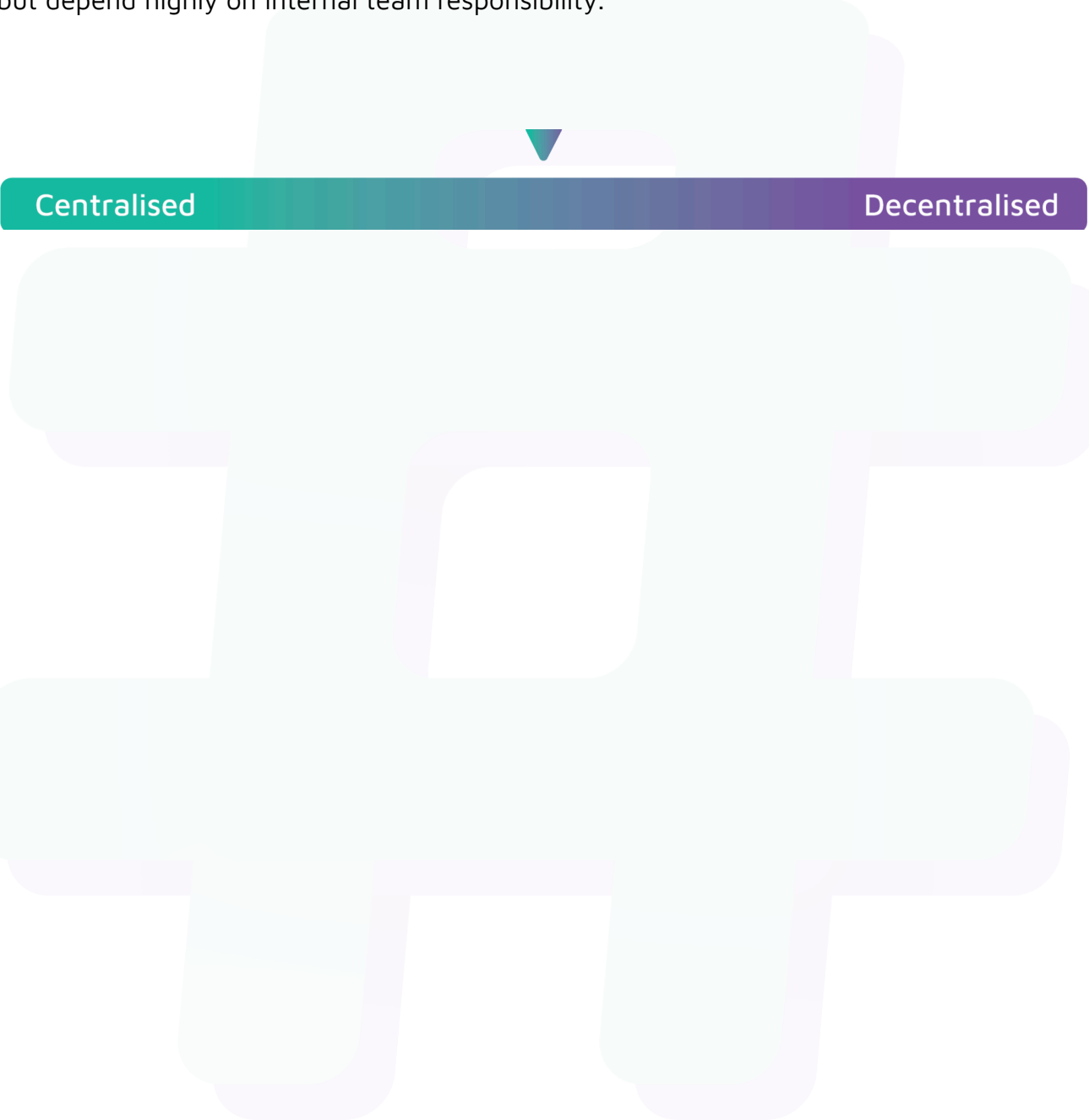
Status

Resolved

Centralisation

The U2U Super App & Mobile wall values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

Conclusion

After Hashlocks analysis, the U2U Super App & Mobile wallet seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our Penetration Tests a collaborative effort. The objective of our security Penetration Tests is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security Penetration Test process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future Penetration Tests. Although our primary focus is on the in-scope code, we examine dependency code and behavior when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the code base under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other Penetration Test results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we still need to verify the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown not to represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed the code bases in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the code base source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the Penetration Test makes no statements or warranties on the security of the code. It also cannot be considered a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this code base.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Code is deployed and executed on a platform. The platform, its programming language, and other software related to the code base can have their own vulnerabilities that can lead to attacks. Thus, the Penetration Test can't guarantee the explicit security of the Penetration Tested code bases.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.